

Proyecto Sistemas Distribuidos

Integrantes del grupo:

María Noel Balado

Camila García

Telma Martinez

Julieta Romero

XX/XX/2026

Índice

1. Introducción..... 2
1.1) Idea..... 2
1.2) Relevancia..... 3
1.3) Uso de la Inteligencia Artificial..... 4
2. Diseño..... 4
2.1) Características de los Sistemas Distribuidos..... 4
2.2) Arquitectura..... 5
2.3) ¿Cómo se abordan las características de los SD en la arquitectura?..... 7
2.4) Conceptos de 12 Factor involucrados..... 8
3. Tecnologías..... 9
3.1) Tecnologías utilizadas y motivo..... 9
3.2) Sistema de almacenamiento permanente utilizado..... 11
4. Comunicación..... 11
4.1) Comunicación síncrona mediante servicios REST..... 11
4.2) Comunicación asíncrona mediante RabbitMQ..... 12
4.3) Justificación de los métodos de comunicación..... 14
5. Observabilidad..... 15
5.1) Implementación..... 16
5.2) Casos demostrables..... 17
5.3) Beneficios..... 18
5.4) Limitaciones y trabajo futuro..... 18

1. Introducción

1.1) Idea

Problemática

Encontrar una plaza de estacionamiento disponible puede resultar complicado, especialmente en zonas con alta demanda. Los usuarios pueden tener que recorrer diferentes estacionamientos para encontrar una plaza libre, lo que genera pérdida de tiempo, tránsito innecesario y falta de información sobre la disponibilidad real de los espacios.

Además, la disponibilidad de las plazas puede cambiar constantemente, por lo que una información estática o desactualizada resulta insuficiente. Es necesario disponer de un sistema capaz de recibir información en tiempo real sobre el estado de las plazas y actualizarla para los usuarios.

Solución propuesta

El proyecto propone un sistema de estacionamiento inteligente: una plataforma que permite consultar la disponibilidad de plazas en distintos estacionamientos, crear una reserva y cancelarla si el usuario cambia de opinión.

La ocupación de las plazas se actualizará mediante sensores simulados, que publicarán eventos cuando una plaza se ocupe o se libere. El servicio de Parkings recibirá estos eventos y actualizará la información correspondiente. Debido al procesamiento asíncrono, puede existir un breve retraso entre el cambio detectado por el sensor y su reflejo en la disponibilidad consultada por los usuarios.

El sistema distinguirá entre la ocupación física de una plaza y su estado de reserva. Una plaza físicamente libre puede estar reservada, por lo que la disponibilidad se determinará considerando ambos aspectos. De esta manera, un evento del sensor que indique que una plaza está libre no eliminará una reserva vigente.

La gestión de reservas se coordinará con el servicio de Parkings para verificar y asignar la plaza mediante una operación atómica, evitando que dos solicitudes concurrentes obtengan la misma plaza. Al cancelar una reserva, se liberará su asignación; la plaza volverá a estar disponible si también se encuentra físicamente libre.

Alcance delimitado

Para mantener el proyecto dentro de un alcance viable, se establecen las siguientes funcionalidades y limitaciones:

- Se trabajará con un número limitado de estacionamientos y plazas.
- Se incluirán el registro e inicio de sesión de usuarios, la consulta de disponibilidad y la creación y cancelación de reservas.

- Las reservas corresponderán a la asignación de plazas disponibles en el momento de la solicitud. No se contemplarán reservas anticipadas por franjas horarias.
- Los sensores serán simulados, y generarán eventos de ocupación y liberación de plazas, sin utilizar hardware físico.
- Se distinguirá entre la ocupación física de una plaza y su estado de reserva para determinar su disponibilidad.
- Se incluirá un servicio de notificaciones para procesar eventos relacionados con las reservas y los cambios de disponibilidad. La entrega de mensajes se realizará mediante un proveedor externo o una integración simulada para las pruebas.
- Se recopilarán métricas básicas de ocupación y reservas, que podrán consultarse mediante un dashboard.
- No se implementará el pago del estacionamiento. El servicio de Pagos, su base de datos y la pasarela externa se incluyen en el diagrama únicamente como una posible ampliación futura.
- No se gestionará el acceso físico al estacionamiento mediante barreras o cámaras.
- Los servicios y sus dependencias locales se ejecutarán mediante contenedores Docker, utilizando Docker Compose para definir y ejecutar el entorno de prueba.
- La arquitectura permitirá evaluar la comunicación síncrona y asíncrona, la gestión de solicitudes concurrentes y la recuperación ante interrupciones temporales de los consumidores de eventos.
- El escalado se evaluará de forma manual en servicios seleccionados. No se implementará autoescalado. El firewall y el balanceador de carga se representan como elementos del diseño de infraestructura, cuya configuración productiva queda fuera del alcance del prototipo.

1.2) Relevancia

El proyecto resulta relevante desde el punto de vista de los sistemas distribuidos, ya que una aplicación de estacionamiento inteligente requiere coordinar servicios independientes que intercambian información mediante la red y administran sus propios datos.

La disponibilidad de las plazas cambia dinámicamente debido a los eventos generados por los sensores, mientras que simultáneamente diferentes usuarios pueden consultar y reservar plazas. Esto permite abordar problemas de concurrencia y consistencia, como evitar que dos solicitudes obtengan la misma plaza y mantener una disponibilidad que considere tanto la ocupación física como las reservas vigentes.

La combinación de comunicación síncrona mediante REST y comunicación asíncrona mediante RabbitMQ permite utilizar distintos mecanismos según las necesidades de cada operación. Las solicitudes de los usuarios requieren una respuesta inmediata, mientras que las actualizaciones destinadas a notificaciones y métricas pueden procesarse de forma asíncrona.

El servicio de Métricas y el dashboard complementan la gestión de estacionamientos al permitir visualizar indicadores de ocupación y reservas a partir de los eventos del sistema. Esto permite analizar cómo la información generada por distintos servicios puede integrarse sin acceder directamente a sus bases de datos.

Finalmente, la separación del sistema en diferentes servicios permite evaluar el escalado manual de componentes seleccionados y observar el comportamiento del sistema ante interrupciones temporales de los consumidores de eventos. De esta manera, el proyecto ofrece un escenario para estudiar concurrencia, comunicación entre procesos, consistencia, escalabilidad y tolerancia a fallos.

1.3) Uso de la Inteligencia Artificial

Metodología de uso y decisiones en las que participó. Agregar al final. Detallar de qué manera se utilizó la IA durante la ejecución del proyecto y enumerar las decisiones concretas en las que la IA sirvió como herramienta de apoyo

- *Se utilizó la IA para definir la idea de proyecto*
- *Características y 12 factor es propio!!*
- *Sugerencia de que pilar de observabilidad utilizar*

2. Diseño

2.1) Características de los Sistemas Distribuidos

De las características vistas en el curso (según Tanenbaum y Coulouris), se considera que las más relevantes para esta problemática son:

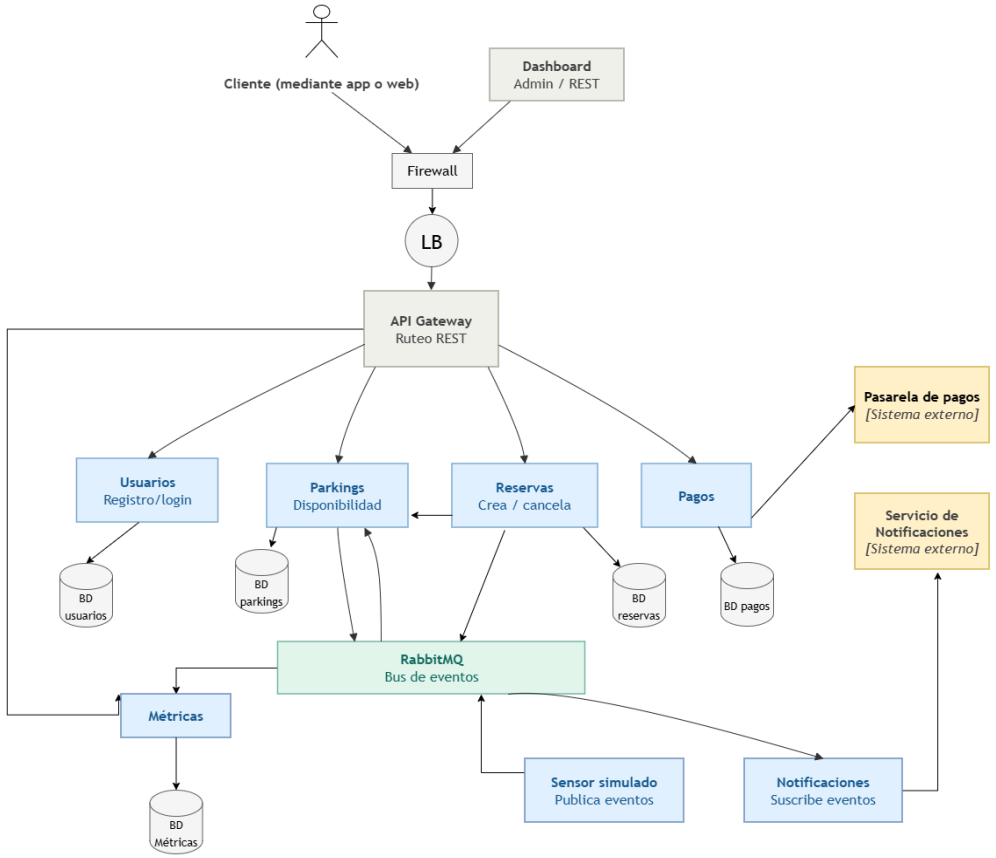
1. **Transparencia (de acceso y de locación):** el usuario interactúa con una sola interfaz y una API común, sin necesitar conocer las direcciones de los servicios internos ni los detalles de cómo estos acceden a sus datos. El API Gateway centraliza el acceso y dirige las solicitudes al servicio correspondiente.
2. **Escalabilidad (en tamaño):** el sistema debe contemplar el crecimiento de usuarios, estacionamientos y sensores. La separación en servicios permite aumentar las instancias de componentes seleccionados según la demanda. En el prototipo, el escalado se evaluará de forma manual, considerando la distribución de solicitudes y las posibles limitaciones de las bases de datos y del broker de mensajería. El escalado manual permite demostrar escalabilidad horizontal sin sumar toda la infraestructura de automatización.
3. **Concurrencia:** distintos usuarios pueden consultar y solicitar reservas simultáneamente, incluso sobre una misma plaza. Además, estas solicitudes pueden coincidir con eventos de los sensores. El sistema debe coordinar estas operaciones para evitar asignaciones duplicadas y mantener una disponibilidad coherente con la ocupación física y las reservas vigentes.

4. **Tolerancia a fallos:** los componentes pueden sufrir interrupciones temporales o problemas de comunicación. La arquitectura debe reducir el impacto de estos fallos sobre las funcionalidades que no dependen del componente afectado. En particular, se evaluará que los eventos destinados a consumidores temporalmente fuera de servicio permanezcan pendientes y puedan procesarse cuando estos se recuperen.
5. **Apertura:** la comunicación entre componentes se basa en interfaces definidas y documentadas, mediante endpoints REST y contratos de eventos. Estos contratos establecen las operaciones disponibles, los formatos de los mensajes y su significado, facilitando la integración y evolución de los servicios.
6. **Heterogeneidad:** la arquitectura permite integrar componentes con tecnologías diferentes, como servicios de aplicación, bases de datos, RabbitMQ y proveedores externos. La comunicación mediante protocolos y contratos acordados permite que estas diferencias tecnológicas no tengan que ser conocidas por los demás componentes.

2.2) Arquitectura

Para representar la arquitectura de la solución se realizó un AOD (Architectural Overview Diagram) que muestra los principales componentes del sistema, sus responsabilidades y las formas de comunicación entre ellos.

Figura 1. Arquitectura general del sistema de estacionamiento inteligente.



Se optó por una arquitectura basada en microservicios, combinada con una arquitectura orientada a eventos (EDA) mediante RabbitMQ. Las operaciones que necesitan conocer un resultado para continuar utilizan HTTP/REST, mientras que los eventos permiten comunicar cambios sin esperar a que los consumidores completen su procesamiento.

La separación en servicios permite distribuir responsabilidades, escalar componentes según su carga e incorporar nuevos consumidores de eventos. También reduce el impacto de la interrupción de servicios como Notificaciones o Métricas sobre las operaciones principales. Como contrapartida, introduce la necesidad de coordinar servicios independientes y contemplar fallos de comunicación.

Componentes y responsabilidades

- **Usuarios**: administra el registro, la autenticación y los datos de los usuarios.
- **Parkings**: administra los estacionamientos y sus plazas. Mantiene la ocupación física y la asignación de cada plaza, determina su disponibilidad y procesa los eventos de los sensores. También publica eventos cuando cambia la disponibilidad.
- **Reservas**: administra la creación y cancelación de reservas y conserva su información. Se comunica con Parkings para solicitar la asignación o liberación de una plaza y publica los eventos correspondientes a los cambios de las reservas.
- **Notificaciones**: consume los eventos pertinentes y solicita la entrega de avisos mediante un proveedor externo o una integración simulada.
- **Métricas**: consume eventos de disponibilidad y reservas, calcula indicadores y almacena los resultados necesarios para su consulta.
- **Dashboard administrativo**: presenta indicadores de ocupación y reservas. Consulta la API de Métricas mediante el Gateway.
- **Sensor simulado**: genera eventos de ocupación y liberación de plazas, que publica en RabbitMQ.
- **RabbitMQ**: recibe y enruta los eventos hacia las colas de los servicios interesados.
- **API Gateway**: centraliza el acceso de la aplicación y del dashboard, y dirige las solicitudes hacia los servicios correspondientes.

El servicio de Pagos, su base de datos y la pasarela externa se representan únicamente como una posible ampliación futura. No forman parte de la implementación del prototipo.

Comunicación entre componentes

La aplicación y el dashboard se comunican con el API Gateway mediante HTTP/REST. También se utiliza REST entre Reservas y Parkings para coordinar la asignación y liberación de plazas. Parkings determina si una plaza puede asignarse considerando su ocupación física, la existencia de una reserva vigente y la vigencia de la información del sensor.

La comunicación asíncrona se realiza mediante RabbitMQ. Los sensores publican eventos y reportes que consume Parkings; Parkings publica cambios de disponibilidad y vigencia de la información; y Reservas publica la creación y cancelación de reservas. Métricas y Notificaciones consumen los eventos que necesitan para sus responsabilidades. Los productores no esperan a que los consumidores terminen de procesarlos, aunque pueden recibir confirmaciones de publicación del broker. Los mecanismos de comunicación y recuperación se desarrollan en la sección 4 “Comunicación”.

Despliegue e infraestructura

Los servicios y sus dependencias locales se ejecutarán en contenedores Docker. Docker Compose permitirá definir y ejecutar el entorno de prueba, incluyendo sus conexiones y volúmenes de persistencia. El escalado se evaluará manualmente en componentes seleccionados.

La arquitectura contempla un firewall para restringir el tráfico autorizado y un balanceador para distribuir solicitudes entre instancias del Gateway cuando estas se desplieguen. Estos elementos representan el diseño de infraestructura; su configuración productiva y el autoescalado quedan fuera del alcance del prototipo.

2.3) ¿Cómo se abordan las características de los SD en la arquitectura?

- **Transparencia de acceso y de locación**: el API Gateway ofrece un punto de entrada común para la aplicación y el dashboard. Los clientes no necesitan conocer las direcciones de los servicios internos ni cómo se distribuyen sus instancias.
- **Escalabilidad**: la separación en servicios y su ejecución en contenedores permiten replicar componentes de manera independiente. En el prototipo, el escalado será manual: las solicitudes REST requerirán distribución entre réplicas, mientras que las instancias consumidoras podrán compartir una cola de RabbitMQ para repartir el trabajo. La capacidad del broker y de las bases de datos también puede limitar el escalado.
- **Concurrencia**: Parkings protege la asignación de plazas mediante una actualización condicional y atómica en PostgreSQL. Solo se asigna una plaza si está físicamente libre, no tiene una asignación vigente y dispone de información actualizada del sensor. Así se evitan asignaciones duplicadas ante solicitudes concurrentes. La coordinación con Reservas requiere, además, identificadores de operación, reintentos idempotentes y recuperación de operaciones incompletas.
- **Tolerancia a fallos**: RabbitMQ permitirá conservar eventos pendientes mediante colas durables, mensajes persistentes y confirmaciones de publicación y procesamiento. Los consumidores deberán contemplar entregas duplicadas. La interrupción de Métricas o Notificaciones no impedirá las operaciones principales, ya que estas no esperan su procesamiento. Además, Parkings detectará la ausencia de reportes periódicos y marcará la información afectada como desactualizada,

impidiendo nuevas asignaciones hasta recibir información vigente, sin eliminar reservas existentes.

- **Apertura**: los endpoints REST y los contratos de eventos definen las operaciones, los formatos y el significado de los mensajes. Esto facilita integrar o reemplazar componentes e incorporar consumidores, siempre que se mantenga la compatibilidad de las interfaces.
- **Heterogeneidad**: los protocolos y contratos acordados permiten integrar servicios de aplicación, PostgreSQL, RabbitMQ y proveedores externos. Cada componente puede utilizar tecnologías diferentes sin exponer sus detalles internos a los demás.

2.4) Conceptos de 12 Factor involucrados

A partir de las decisiones arquitectónicas adoptadas para el sistema, se evalúa la aplicación de la metodología 12 Factor

1. **Configuración (Factor 3):** La configuración del sistema, que varía entre los diferentes entornos de desarrollo, pruebas o producción (tales como credenciales de conexión a las bases de datos de Pagos y Reservas, las rutas de comunicación del API Gateway, o los parámetros de conexión con el broker RabbitMQ), se almacena en variables de entorno. Esto asegura una estricta separación entre el código y la configuración, permitiendo modificar los parámetros de infraestructura sin necesidad de recompilar los artefactos de los servicios.
2. **Backing service (Factor 4):** Los servicios de respaldo se tratan como recursos externos adjuntos. El sistema desacopla la lógica de negocio de su infraestructura de mensajería al utilizar RabbitMQ como un bus de eventos externo para la comunicación asíncrona, así como bases de datos propias para Usuarios, Parkings, Reservas y Métricas. Estos recursos se accederán mediante direcciones y credenciales definidas en la configuración. Esto permitirá, por ejemplo, sustituir una instancia local de PostgreSQL o RabbitMQ por otra compatible alojada en un servidor diferente, modificando la configuración sin alterar el código de los microservicios
3. **Procesos (Factor 6):** Los microservicios se ejecutan como procesos independientes, sin estado en su memoria local para la persistencia del flujo principal. Esto se evidencia en el diseño orientado a eventos: si un consumidor (como el servicio de Notificaciones o Métricas) cae temporalmente, los eventos no se pierden, sino que quedan encolados en el broker de mensajes hasta que el proceso se recupere y retome su ejecución de manera autónoma.
4. **Asignación de puertos (Factor 7):** Los servicios son completamente autónomos en su exposición en la red. El sistema utiliza un API Gateway como punto único de entrada que enruta las solicitudes HTTP/REST hacia los microservicios correspondientes, los cuales operan de forma independiente exponiendo sus propias interfaces de comunicación en red.
5. **Concurrencia (Factor 8):** La arquitectura escala mediante el modelo de procesos horizontales. La separación de responsabilidades en microservicios independientes

permite dimensionar y escalar cada componente de forma aislada según su carga de trabajo específica. Por ejemplo, el servicio de Parkings puede escalar horizontalmente para soportar el alto volumen de lecturas y eventos de sensores, de manera totalmente independiente de los servicios de menor frecuencia transaccional como Pagos.

6. **Logs (Factor 11):** El sistema sigue el principio de tratar los logs como un flujo continuo de eventos generados por los procesos de la aplicación. Los microservicios escribirán sus registros en la salida estándar (stdout) y en la salida de errores (stderr), en lugar de gestionar archivos de registro propios. En el entorno de prueba, Docker permitirá consultar estos registros mediante los comandos correspondientes, como docker compose logs. Esta solución facilita revisar el comportamiento de los servicios y diagnosticar posibles errores sin incorporar inicialmente una plataforma adicional de centralización. La recopilación centralizada, el análisis avanzado y la conservación de los registros a largo plazo se consideran posibles mejoras futuras.

Otros cambios para el proyecto:

***Detección de sensores sin conexión**: En el dashboard se podría mostrar plazas libres, ocupadas, reservadas y con información desactualizada, además de generar una alerta cuando un sensor deja de reportar. Permite demostrar que hace el sistema cuando deja de recibir información en lugar de asumir que todos los sensores funcionan siempre*.

3. Tecnologías

3.1) Tecnologías utilizadas y motivo

La tecnologías utilizadas para la implementación del proyecto fueron las siguientes:

Tabla 1. Tabla de tecnologías principales utilizadas para el sistema de estacionamiento inteligente.

Tecnología propuesta	Uso concreto en el proyecto	Motivo de elección
Python	Implementación de Usuarios, Parkings, Reservas, Notificaciones, Métricas y el simulador de sensores.	Permite compartir herramientas y reducir la variedad de lenguajes que el equipo debe mantener.
FastAPI + Uvicorn	FastAPI para las APIs REST y Uvicorn para ejecutarlas. Los consumidores de eventos pueden ejecutarse como procesos Python independientes.	Facilita definir y validar solicitudes, documentar endpoints y probar las APIs.

SQLAlchemy + psycopg	Acceso desde los servicios Python a PostgreSQL.	Permite gestionar persistencia y transacciones, incluyendo actualizaciones condicionales para asignar plazas.
PostgreSQL	Bases de datos propias de Usuarios, Parkings, Reservas y Métricas. En el prototipo pueden alojarse en una misma instancia, manteniendo bases y credenciales separadas.	Ofrece persistencia relacional y transacciones. Cada servicio conserva el control de sus datos.
RabbitMQ mediante AMQP	Transporte de eventos de sensores, disponibilidad, reservas y vigencia de la información.	Permite enrutar mensajes, mantener colas independientes y distribuir trabajo entre consumidores.
Pika	Cliente de RabbitMQ para los procesos Python que publican y consumen eventos.	Permite implementar la comunicación AMQP y gestionar confirmaciones de publicación y procesamiento.
NGINX Open Source	Implementación del API Gateway mediante rutas como /usuarios, /parkings, /reservas y /metricas.	Permite centralizar el acceso y dirigir solicitudes a los servicios. Para las pruebas de escalado puede configurarse la distribución hacia réplicas.
TypeScript + React + Next.js	Aplicación web de usuarios y dashboard administrativo dentro de un mismo proyecto, con pantallas diferenciadas.	Permite reutilizar componentes, organizar las páginas y definir tipos para los datos intercambiados con las APIs.
CSS Modules	Estilos de la aplicación y del dashboard.	Permite mantener estilos asociados a los componentes sin agregar otra biblioteca de interfaz inicialmente.
Docker	Contenedores para frontend, servicios, Gateway y dependencias.	Facilita reproducir el entorno y controlar sus dependencias.
Docker Compose	Definición del entorno local, redes, configuración y volúmenes.	Permite ejecutar conjuntamente los componentes del prototipo.

Logging de Python y registros de NGINX	Registro de errores, solicitudes, procesamiento de eventos y pérdida o recuperación de reportes.	Aporta evidencia para investigar fallos y complementar los indicadores del dashboard.

3.2) Sistema de almacenamiento permanente utilizado

4. Comunicación

Para la comunicación entre los componentes del sistema decidimos que se utilizarán dos mecanismos: servicios REST mediante HTTP para la comunicación síncrona y mensajería asíncrona mediante RabbitMQ. Ambos mecanismos se complementan para responder a las necesidades de la arquitectura distribuida, permitiendo tanto la ejecución de operaciones que requieren una respuesta inmediata como la propagación de eventos que pueden procesarse de manera independiente.

4.1) Comunicación síncrona mediante servicios REST

REST (*Representational State Transfer*) es un estilo arquitectónico para diseñar servicios que, en este proyecto, se implementará sobre HTTP. Organiza la información en recursos identificados mediante URL y permite interactuar con ellos utilizando métodos como GET, POST, PUT, PATCH y DELETE.

Una API REST define las operaciones que un servicio ofrece a otros procesos remotos, junto con el formato de los mensajes y las reglas necesarias para utilizarlas.

En estas interacciones se utiliza un modelo de solicitud y respuesta: un servicio actúa como servidor, exponiendo sus operaciones, mientras que otro componente actúa como cliente, invocándolas y esperando un resultado. Un mismo servicio puede asumir ambos roles en distintas interacciones.

En este sistema, la aplicación y el dashboard administrativo se comunican con el API Gateway mediante HTTP/REST. El Gateway dirige las solicitudes hacia los servicios correspondientes. Además, se utiliza comunicación REST directa entre Reservas y Parkings para coordinar la asignación o liberación de una plaza.

Los principales casos de uso son:

- **Consulta de disponibilidad:** la aplicación realiza una solicitud GET para recuperar la información de las plazas disponibles.
- **Creación de reservas:** la aplicación envía una solicitud POST al servicio de Reservas.
- **Cancelación de reservas:** la aplicación solicita la cancelación de una reserva existente mediante el método HTTP correspondiente al diseño de la API.

- **Asignación de plazas:** Reservas invoca una operación del servicio de Parkings para solicitar la asignación de una plaza.
- **Liberación de plazas:** al cancelar una reserva, Reservas solicita a Parkings que libere la asignación correspondiente.
- **Consulta de métricas y alertas:** el dashboard realiza solicitudes GET, mediante el Gateway, al servicio de Métricas para obtener indicadores de ocupación y reservas, así como información sobre sensores sin reportes recientes y plazas con información desactualizada**.**

La elección de REST se justifica porque estas operaciones requieren intercambiar solicitudes y respuestas con un resultado definido. El cliente espera la respuesta del servidor para determinar cómo continuar, por ejemplo, para informar al usuario si una reserva se creó correctamente o si la plaza ya no se encuentra disponible.
En particular, la asignación de plazas debe realizarse mediante una operación atómica en el servicio de Parkings. Este verifica que la plaza se encuentre físicamente libre y que no tenga una asignación vigente antes de confirmar la operación. Si dos solicitudes concurrentes intentan reservar la misma plaza, solo una podrá obtener la asignación; la otra recibirá una respuesta de conflicto. De esta forma, la decisión no depende únicamente de una consulta previa de disponibilidad.

Como contrapartida, la comunicación síncrona genera una dependencia en el tiempo entre cliente y servidor: si el servicio requerido no responde, la solicitud puede fallar o quedar esperando hasta que se alcance un tiempo de espera. Además, las latencias de las llamadas consecutivas pueden acumularse. Por este motivo, REST se reserva para las operaciones que necesitan conocer el resultado de la interacción para continuar.

4.2) Comunicación asíncrona mediante RabbitMQ
RabbitMQ es un *message broker* que permite la comunicación asíncrona entre aplicaciones mediante el intercambio y enrutamiento de mensajes. En este sistema se utilizará el protocolo AMQP para publicar eventos y distribuirlos entre los servicios interesados.

El productor publica un mensaje en el broker y continúa su ejecución sin esperar a que los consumidores terminen de procesarlo. RabbitMQ recibe el mensaje y lo enruta hacia las colas correspondientes, donde queda disponible para los consumidores.

En este mecanismo intervienen los siguientes elementos:
- **Productor (Producer):** componente que genera y publica un mensaje. En el sistema, los sensores simulados producen eventos de ocupación y liberación, mientras que Reservas produce eventos de creación y cancelación de reservas.
- **Exchange:** recibe los mensajes publicados y determina a qué colas deben dirigirse según sus reglas de enrutamiento.

- **Routing key:** clave que permite clasificar los mensajes y dirigirlos hacia las colas correspondientes.
- **Cola (Queue):** almacena los mensajes pendientes de procesamiento.
- **Consumidor (Consumer):** servicio que recibe y procesa los mensajes de una cola.

El productor no necesita conocer qué servicios consumirán sus eventos ni cuántos consumidores existen. Solo publica el mensaje en RabbitMQ, que se encarga de distribuirlo. Esto reduce el acoplamiento entre los componentes y permite incorporar o modificar consumidores sin tener que cambiar el productor, siempre que se mantenga el contrato de los eventos.

En este sistema, RabbitMQ se utilizará para los siguientes casos de uso:

- **Actualización de ocupación:** los sensores simulados publican eventos cuando una plaza se ocupa o se libera. El servicio de Parkings consume estos eventos para actualizar el estado físico de las plazas.
- **Reportes periódicos de sensores**: los sensores simulados también publican reportes con su estado actual, aunque la ocupación no cambie. Parkings registra la última comunicación recibida de cada sensor y comprueba periódicamente si se supera el tiempo límite definido sin recibir nuevos reportes.
- **Eventos de disponibilidad**: Parkings publica eventos cuando cambia la disponibilidad de una plaza, considerando su ocupación física, su asignación y la vigencia de la información del sensor.
- **Eventos de información desactualizada y recuperación**: cuando se supera el tiempo límite sin reportes, Parkings marca la información como desactualizada y publica el evento correspondiente. Cuando vuelve a recibir información vigente, actualiza ese estado y publica su recuperación. La ausencia de reportes indica que no se dispone de información reciente, sin afirmar necesariamente que el sensor esté desconectado.
- **Eventos de reservas:** Reservas publica eventos cuando se crea o cancela una reserva.
- **Notificaciones:** el servicio de Notificaciones consume los eventos pertinentes para generar los avisos correspondientes a los usuarios.
- **Actualización de indicadores y alertas:** Métricas consume eventos de disponibilidad, reservas y cambios en la vigencia de la información para actualizar los indicadores y las alertas que posteriormente consulta el dashboard.

Cuando varios servicios necesitan procesar el mismo evento, se configurarán colas independientes para cada consumidor interesado. De esta manera, un evento de reserva puede ser procesado tanto por Notificaciones como por Métricas sin que un servicio dependa de que el otro termine su trabajo. Las distintas instancias de un mismo servicio podrán compartir una cola para distribuir los mensajes y permitir el escalado horizontal.

La elección de RabbitMQ se justifica por el desacoplamiento temporal entre productores y consumidores, la posibilidad de absorber picos de carga mediante las colas y la capacidad de escalar los consumidores de forma independiente. Asimismo, si un consumidor deja de funcionar temporalmente, los mensajes pueden permanecer pendientes hasta que vuelva a estar disponible, siempre que se hayan configurado los mecanismos de persistencia correspondientes.

Para mejorar la confiabilidad, se prevé utilizar colas durables y mensajes persistentes, junto con confirmaciones de publicación (*publisher confirms*) para verificar que el broker haya recibido los mensajes. Los consumidores utilizarán confirmaciones de procesamiento (*acknowledgements* o ACK) una vez completado el trabajo. Si se produce un fallo antes de confirmar el procesamiento, el mensaje podrá volver a entregarse. Por ello, los consumidores deberán contemplar la posibilidad de recibir mensajes duplicados mediante un procesamiento idempotente (procesar el mismo mensaje varias veces produce el mismo resultado final que procesarlo una sola vez). Los mensajes que fallen repetidamente podrán derivarse a una cola de mensajes fallidos (*Dead Letter Queue*, DLQ) para su inspección y recuperación.

Además, los mensajes de los sensores incluirán información que permita identificar su orden y antigüedad, para evitar que un reporte atrasado sobrescriba un estado más reciente o haga considerar vigente información que ya está desactualizada.

4.3) Justificación de los métodos de comunicación

La combinación de REST y RabbitMQ permite seleccionar el mecanismo de comunicación de acuerdo con los requisitos de cada operación.

REST se utiliza cuando un componente necesita invocar una operación ofrecida por otro servicio y conocer su resultado antes de continuar. RabbitMQ se utiliza cuando un componente debe comunicar un acontecimiento a otros servicios sin depender de que estos lo procesen inmediatamente. Esta distinción resulta especialmente importante en la gestión de reservas.

La asignación y liberación de plazas se coordinan mediante llamadas REST para obtener una respuesta sobre el resultado de la operación. En cambio, los eventos destinados a notificaciones y métricas se procesan de forma asíncrona, evitando que las operaciones principales deban esperar a que estas tareas finalicen.

El procesamiento asíncrono permite mantener actualizaciones con consistencia eventual, por lo que puede existir un retraso entre un cambio físico detectado por un sensor y su reflejo en la información consultada, así como en las notificaciones y los indicadores. Además, el sistema distingue entre la ocupación física, el estado de reserva y la vigencia de

la información de cada plaza. Por lo tanto, un evento que indique que una plaza está libre no debe eliminar una reserva vigente, y la cancelación de una reserva solo permitirá que la plaza vuelva a estar disponible si también se encuentra físicamente libre y dispone de información vigente del sensor.

Los reportes periódicos de los sensores también se transmiten mediante RabbitMQ, ya que no requieren una respuesta inmediata de los consumidores. Parkings utiliza estos reportes para detectar información desactualizada y publica eventos cuando cambia su vigencia. Esto permite actualizar los indicadores y las alertas del dashboard sin incorporar su procesamiento a las operaciones de reserva. La falta de reportes se interpreta como ausencia de información reciente, sin asumir necesariamente que el sensor esté desconectado.

Por último, dado que Reservas y Parkings mantienen sus propias bases de datos, las operaciones sobre ambos servicios no forman parte de una única transacción. Será necesario contemplar posibles fallos entre las llamadas mediante identificadores de operación, reintentos idempotentes y mecanismos de recuperación de operaciones incompletas.

En conclusión, REST y RabbitMQ cumplen funciones complementarias dentro de la arquitectura: REST permite realizar operaciones mediante solicitudes y respuestas, mientras que RabbitMQ permite publicar, enrutar y procesar eventos de manera asíncrona y desacoplada. Esta combinación favorece la coordinación de las operaciones críticas, la escalabilidad de los consumidores y la tolerancia a fallos del sistema.

5. Observabilidad

La observabilidad es la capacidad de comprender el estado interno de un sistema a partir de sus salidas externas: logs, métricas y trazas. En este sistema es especialmente importante, porque una misma operación del usuario atraviesa varios componentes (API Gateway, Reservas, Parkings) y parte de la información viaja de forma asíncrona por RabbitMQ, por lo que un error puede manifestarse en un servicio distinto de aquel donde se originó.

Para el prototipo, se priorizarán los logs (registros) para reconstruir el recorrido de las operaciones y analizar errores. Su implementación se apoyará en la infraestructura existente de Docker y en el Factor 11 de la metodología 12-Factor. Además, se incorporará el seguimiento de los reportes de sensores para visualizar la vigencia de la información y detectar interrupciones en su actualización.

5.1) Implementación

Registros de los servicios

Los servicios emitirán logs de los acontecimientos relevantes: recepción y resultado de solicitudes, asignación y liberación de plazas, publicación y procesamiento de eventos, errores y recuperación de operaciones.

Se utilizará un formato estructurado y consistente, preferentemente JSON, con los siguientes campos según corresponda:

- **Fecha y hora**: momento en que se registra el acontecimiento.
- **Servicio e instanci**a: componente que genera el registro.
- **Nivel**: INFO para operaciones normales, WARNING para situaciones que requieren atención y ERROR para fallos.
- **Descripción y resultado**: operación realizada y su resultado.
- **Identificador de correlación**: permite relacionar registros de una misma operación entre servicios.
- **Identificadores asociados**: reserva, plaza, sensor, operación o evento involucrado.
- **Información del error**: causa conocida y número de intento, cuando exista un reintento.
- Se evitará registrar contraseñas, tokens y otros datos sensibles.

Correlación entre componentes

Cada solicitud recibida por el Gateway tendrá un identificador de correlación que se propagará en las llamadas HTTP a los servicios internos. Cuando esa operación produzca eventos, el identificador también se incluirá en sus metadatos para relacionar el procesamiento asíncrono con la solicitud original.

Cada evento tendrá, además, un identificador propio que permita reconocer entregas repetidas. En los eventos generados por sensores se incluirán el identificador del sensor y la información necesaria para determinar el orden y la antigüedad de sus reportes.

Esta correlación facilitará reconstruir el recorrido de una operación mediante los logs, aunque no constituye por sí sola una implementación de trazas distribuidas.

Consulta de los registros

Los procesos escribirán sus registros en la salida estándar, siguiendo el Factor 11 de 12-Factor, que trata los logs como un flujo de eventos y delega su almacenamiento y gestión al entorno de ejecución.

Docker recogerá estas salidas y permitirá consultarlas mediante docker compose logs, utilizando filtros por servicio, marcas de tiempo y seguimiento en tiempo real. Para el

entorno de prueba, esto permitirá revisar los registros sin incorporar una plataforma adicional de centralización.

Seguimiento de sensores
Los sensores simulados enviarán reportes periódicos con su estado actual, aunque no cambie la ocupación. Parkings registrará la última comunicación de cada sensor y comprobará si transcurre un tiempo límite sin recibir información vigente. El intervalo de reporte y el tiempo límite se definirán mediante configuración.

Cuando se supere ese límite, Parkings marcará la información de la plaza como desactualizada, registrará la situación y publicará el evento correspondiente. Este estado será independiente de la ocupación física y de la existencia de una reserva: una plaza podrá conservar una reserva vigente y, al mismo tiempo, tener información desactualizada del sensor.

Métricas procesará estos eventos para que el dashboard muestre las plazas afectadas y una alerta de falta de reportes recientes. Mientras la información esté desactualizada, Parkings impedirá nuevas asignaciones sobre esas plazas, sin eliminar reservas existentes.

Cuando se reciban nuevamente reportes vigentes, se registrará y publicará la recuperación para actualizar el dashboard. Un reporte atrasado no deberá hacer que información antigua se considere actualizada.

Este mecanismo permitirá observar la pérdida de actualización, aunque no determinará por sí solo si la causa está en el sensor, en la comunicación o en el procesamiento de mensajes.

5.2) Casos demostrables

Para evaluar el comportamiento del sistema, se proponen las siguientes pruebas. En cada caso se contrastará la información de los logs con las respuestas de las APIs, el estado almacenado y, cuando corresponda, los datos mostrados en el dashboard.

Caso	¿Qué se hace?	¿Qué muestran los logs?
Concurrencia	Dos solicitudes simultáneas intentan reservar la misma plaza	Que no se asigna la misma plaza a dos reservas activas y que se puede consultar el resultado de ambas solicitudes.
Tolerancia a fallos	Se detiene temporalmente	Que el servicio se recupera

	el contenedor de Notificaciones, se generan reservas y se vuelve a levantar	y, si RabbitMQ está configurado para conservar los mensajes y el consumidor los procesa correctamente, que se atienden los eventos que habían quedado pendientes.
Sensor sin reportes	Se interrumpen los reportes de un sensor simulado y se espera a que pase el tiempo límite.	Que la plaza aparece con información desactualizada y que el dashboard muestra la alerta correspondiente.
Recuperación del sensor	Se reanudan los reportes del sensor.Recuperación del sensor	Que el sistema vuelve a recibir información y actualiza el estado cuando los datos vuelven a estar vigentes.

Los registros se contrastarán con las respuestas de las APIs, el estado persistido y la información del dashboard. Así, la demostración no dependerá únicamente de los mensajes escritos en los logs.

5.3) Beneficios

- **Apoyo al diagnóstico**: permite identificar el componente y la operación donde se registró un fallo, y reunir información para investigar su causa.
- **Visibilidad del flujo distribuido**: el identificador de correlación relaciona lo ocurrido en distintos servicios y en el procesamiento de eventos.
- **Evidencia del comportamiento esperado**: permite comprobar el manejo de solicitudes concurrentes y la recuperación del procesamiento después de interrupciones.
- **Visibilidad de la vigencia de los datos**: el seguimiento de sensores evita presentar información antigua como si continuara actualizada.
- **Trazabilidad de operaciones**: permite relacionar la creación y cancelación de reservas con sus identificadores, plazas y momentos de ejecución.
- **Implementación acotada**: aprovecha las salidas de los contenedores y los componentes existentes, sin requerir inicialmente una plataforma adicional de observabilidad.

5.4) Limitaciones y trabajo futuro

La propuesta se centrará en logs y en indicadores de vigencia de la información de los sensores. No incluirá inicialmente trazas distribuidas ni una plataforma centralizada para

buscar registros, analizar tendencias o generar alertas operativas sobre todos los componentes.

Los identificadores de correlación facilitarán relacionar registros, pero no proporcionarán automáticamente una representación del recorrido completo ni de la duración de cada interacción. Además, las marcas de tiempo de distintos servicios no garantizan por sí solas un orden global exacto.

La conservación de los logs dependerá de la configuración de Docker, incluyendo su rotación y retención. Por ello, estos registros no se considerarán una auditoría permanente ni sustituirán los datos de negocio almacenados en las bases de datos.

La detección de sensores sin reportes recientes también tendrá limitaciones: una demora en RabbitMQ o en Parkings puede provocar información desactualizada aunque el sensor continúe funcionando. Asimismo, la alerta del dashboard dependerá de que Métricas y el propio dashboard estén disponibles y hayan procesado el evento correspondiente.

Como trabajo futuro, se podrá incorporar centralización de logs, trazas distribuidas y métricas técnicas como latencia de las APIs, tasa de errores, mensajes pendientes y tiempo de procesamiento. Estas señales permitirían distinguir mejor entre fallos de sensores, comunicación y servicios.